Rounding and Overflow in C++

ISO/IEC JTC1 SC22 WG21 P0105R0 - 2015-09-27

Lawrence Crowl, Lawrence@Crowl.org

Introduction

C++ currently provides relatively poor facilities for controlling rounding. It has even fewer facilities for controlling overflow. The lack of such facilities often leads programmers to ignore the issue, making software less robust than it could be (and should be).

This paper presents the issues and provides some candidate enumerations and operations. The intent of the paper is to gather feedback on support for and direction of future work.

Rounding

Rounding is necessary whenever the resolution of a variable is coarser than the resolution of a value to be placed in that variable.

Current Status

The `numeric_limits` field `round_style` provides information on the style of rounding employed by a type.

```
namespace std {
  enum float_round_style {
    round_indeterminate = -1, // indeterminable
    round_toward_zero = 0, // toward zero
    round_to_nearest = 1, // to the nearest representable value
    round_toward_infinity = 2, // toward [positive] infinity
    round_toward_neg_infinity = 3 // toward negative infinity
  };
}
```

This specification is incomplete in that it fails to specify what happens when the value is equally far from the two nearest representable values.

The standard also says "Specializations for integer types shall return `round_toward_zero`." This requirement is somewhat misleading as a right-shift operation on a two's complement representation does not round toward zero.

Headers `<cfenv>` and `<fenv.h>` provide functions for setting and getting the floating-point rounding mode, `fesetround` and `fegetround`, respectively. The mode is specified via a macro constant:

Constant	Explanation
`FE_DOWNWARD`	rounding towards negative infinity
`FE_TONEAREST`	rounding towards nearest integer
`FE_TOWARDZERO`	rounding towards zero
`FE_UPWARD`	rounding towards positive infinity

Again, the specification is incomplete with respect to `FE_TONEAREST`

Base Requirements

The base requirements on a *round* function are:

- Given a value x and two adjacent representable values $y < z$ such that $y \le x \le z$ then
  - if $x = y$ then $round(x) = y$,
  - if $x = z$ then $round(x) = z$,
  - and otherwise $round(x) = y$ or $round(x) = z$.
- Given an additional value w such that $y \le w \le x \le z$ then
  - $y \le round(w) \le round(x) \le z$

Modes

The number of rounding modes is perhaps unlimited. However, we can explore the space of reasonably efficient rounding modes with two notions, its direction and its domain.

There are six precisely-defined rounding directions and at least three additional practical directions. They are:

towards negative infinity	towards positive infinity
towards zero	away from zero
towards even	towards odd
fastest execution time	smallest generated code
whatever, I'm not picky	

Of these directions, only towards even and towards odd are unbiased.

Rounding towards odd has two desirable properties. First, the direction will not induce a carry out of the units position. This property avoids overflow and increased representation size. Second, because most operations tend to preserve zeros in the lowest bit, the towards-even direction carries less information than towards-odd. This effect increases as the number of bits decreases. However, rounding towards even produces numbers that are "nicer" than those produced by rounding towards odd. For example, you are more likely to get 10 than 9.9999999 with rounding towards even.

There are at least two direction domains:

- All values between two representable values move in the given direction.
- Only values midway between two representable values move in the given direction. Other values move to the nearest representable value. That is, the direction is a tie breaker.

Several of the precise rounding modes are in current use.

direction	**domain**	**compatibility**	**uses**
towards negative infinity	all	`round_toward_neg_infinity` `FE_DOWNWARD`	interval arithmetic lower bound two's complement right shift
	tie		
towards positive infinity	all	`round_toward_infinity` `FE_UPWARD`	interval arithmetic upper bound
	tie		
towards zero	all	`round_toward_zero` `FE_TOWARDZERO`	C/C++ integer division signed-magnitude right shift
	tie		
away from zero	all		
	tie	the `<cmath> round` functions	schoolbook rounding
towards nearest even	all		

	tie	`round_to_nearest` `FE_TONEAREST`	general floating-point computation
towards nearest odd	all		
	tie		some accounting rules best information preservation
fastest			low latency
smallest			low code space
unspecified		`round_indeterminate`	?

We represent the mode in C++ as an enumeration:

```
enum class rounding {
  all_to_neg_inf, all_to_pos_inf,
  all_to_zero, all_away_zero,
  all_to_even, all_to_odd,
  all_fastest, all_smallest,
  all_unspecified,
  tie_to_neg_inf, tie_to_pos_inf,
  tie_to_zero, tie_away_zero,
  tie_to_even, tie_to_odd,
  tie_fastest, tie_smallest,
  tie_unspecified
};
```

The unmotivated modes `all_away_zero`, `all_to_even`, `all_to_odd`, `tie_to_neg_inf`, `tie_to_pos_inf`, and `tie_to_zero` are *conditionally* supported.

Functions

Within the definition of the following functions, we use a defining function, which we do not expect will be directly represented in C++. It is *T round*(*mode,U*) where *U* either

- has a finer resolution than *T* or
- is evaluated as a real number expression.

We already have rounding functions for converting floating-point numbers to integers. However, we need a facility that extends to different sizes of floating-point and between other numeric types.

`template<typename T, typename U> T convert(rounding mode, U value)`

> The result is *round*`(mode, U)`.

`template<rounding mode, typename T, typename U> T convert(U value)`

> The result is `round<T>(mode, U)`.

A division function has obvious utility.

`template<typename T> T divide(rounding mode, T dividend, T divisor)`

> The result is *round*`(mode, dividend/divisor)`. Remember that division evaluates as a real number. Obviously, the implementation will use a different strategy, but it must yield the same result.

`template<rounding mode, typename T> T divide(T dividend, T divisor)`

> The result is `divide(mode, dividend, divisor)`.

Division by a power of two has substantial implementation efficiencies, and is used heavily in fixed-point arithmetic as a scaling mechanism. We represent the conjunction of these approaches with a rounding scale down (right shift).

`template<typename T> T scale_down(rounding mode, T value, int bits)`

> The result is *round*$(\texttt{mode, dividend}/2^{\texttt{bits}})$.

`template<rounding mode, typename T> T scale_down(T value, int bits)`

> The result is `scale_down(mode, dividend, bits)`.

We can add other functions as needed.

Overflow

Overflow is possible whenever the range of an expression exceeds the range of a variable.

Current Status

Signed integer overflow is undefined behavior. Programmers attempting to detect and handle overflow often get it wrong, in that they end up using overflow to detect overflow. Suffice it to say that present solutions are inadequate.

Unsigned integer overflow is defined to be mod $2^{\text{bits-in-type}}$. While this definition is exactly right when coding in modular arithmetic, it is counter-productive when one is using unsigned arithmetic to state that the value is non-negative. In the latter environment, undefined behavior on overflow is better, as it enables tools to detect problems.

Floating-point overflow can be detected and altered via `fegetexceptflag`, `fesetexceptflag`, `feclearexcept`, and `feraiseexcept` with the value `FE_OVERFLOW`. However, such checking requires additional out-of-band effort. That is, any checking takes place in code separate from the operations themselves.

Base Requirements

The base requirements on a *overflow* function are:

- Given a value x and a representable range $y \le z$ such that $y \le x \le z$ then an overflow does not occur and
  - $overflow(x) = x$.
- Otherwise, an overflow has occured and the function may, for all overflow values, choose either:
  - Consider the expression an error, handling it or not as appropriate.
  - Return a normal value $w = overflow(x)$ such that $y \le w \le z$.
  - Return a special value indicating overflow, e.g. IEEE infinities. This choice implies defining the result of operations given this special value as an argument.

Modes

Several overflow modes are possible. We categorize them based on the choices in the base requirements. Other modes may be possible or desirable as well.

Some error modes are as follows.

impossible

: Mathematically, overflow cannot occur. This mode is useful when an overflow specification is necessary, but compiler-based range propogation is insufficient to eliminating a check. The mode is an assertion on the part of the programmer. It invites reviewers to examine the accompanying proof. Ignoring overflow and letting the program stray into undefined behavior is a suitable implementation.

undefined

: The programmer states that overflow is sufficiently rare so that overflow is not a concern. Aborting on overflow is a suitable implementation. So is ignoring the issue and letting the program stray into undefined behavior.

abort

: Abort the program on overflow. Detection is required.

quick_exit

: Call `quick_exit` on overflow. Detection is required.

exception

: Throw an exception on overflow. Detection is required.

A special substitution mode is as follows. Detection is required.

special

Return one of possibly several special values indicating overflow.

Some normal substitution modes are as follows. Detection is required.

saturate

Return the nearest value within the valid range.

modulo with shifted scale

For unsigned arguments and range from 0 to *z*, the result is simply *x* mod (*z*+1). Shifting the range such that $0 < y \le z$ requires a more complicated expression, $y + ((x–y) \bmod (z–y+1))$. We can also use this expression when $y < 0$. That is, it is a general purpose definition. However, it may not yield results consistent with division.

Various overflow modes are in current use.

mode	**uses**
impossible	well-analyzed programs
undefined	C/C++ signed integers C (TR 18037) unsaturated fixed-point types most programs
abort	several run-time checking systems
exception	Ada integers C# integers in checked context
special	IEEE floating-point
saturate	C (TR 18037) unsaturated fixed-point types digital signal processing hardware
modulo with shifted scale	two's-complement wrap-around C/C++ unsigned integers C# integers in unchecked context Java signed integers

We represent the mode in C++ as an enumeration:

```
enum class overflow {
  impossible, undefined, abort, exception,
  special,
  saturate, modulo_shifted
};
```

Functions

Within the definition of the following functions, we use a defining function, which we do not expect will be directly represented in C++. It is *T overflow*(*mode*,*T lower*,*T upper*,*U value*) where *U* either

- has a range that is not a subset of the range of *T* or
- is evaluated as a real number expression.

Many C++ conversions already reduce the range of a value, but they do not provide programmer control of that reduction. We can give programmers control.

```
template<typename T, typename U> T convert(overflow mode, U value)
```

The result is *overflow*`(mode, numeric_limits<T>::min, numeric_limits<T>::max, value)`.

```
template<overflow mode, typename T, typename U> T convert(U value)
```

The result is `convert(mode, value)`.

Being able to specify overflow from a range of values of the same type is also helpful.

```
template<typename T> T limit(overflow mode, T lower, T upper, T value)
```

The result is *overflow*`(mode, lower, upper, value)`.

`template<overflow mode, typename T> T limit(T lower, T upper, T value)`

The result is `limit(mode, lower, upper, value)`.

Common arguments can be elided with convenience functions.

`template<typename T> T limit_nonnegative(overflow mode, T upper, T value)`

The result is `limit(mode, 0, upper, value)`.

`template<overflow mode, typename T> T limit_nonnegative(T upper, T value)`

The result is `limit_nonnegative(mode, upper, value)`.

`template<typename T> T limit_signed(overflow mode, T upper, T value)`

The result is `limit(mode, -upper, upper, value)`.

`template<overflow mode, typename T> T limit_signed(T upper, T value)`

The result is `limit_signed(mode, upper, value)`.

Two's-complement numbers are a slight variant on the above.

`template<typename T> T limit_twoscomp(overflow mode, T upper, T value)`

The result is `limit(mode, -upper-1, upper, value)`.

`template<overflow mode, typename T> T limit_twoscomp(T upper, T value)`

The result is `limit_twoscomp(mode, upper, value)`.

For binary representations, we can also specify bits instead. While this specification may seem redundant, it enables faster implementations.

`template<typename T> T limit_nonnegative_bits(overflow mode, T upper, T value)`

The result is *overflow*`(mode, 0,` $2^{\texttt{upper}}$`-1, value)`.

`template<overflow mode, typename T> T limit_nonnegative_bits(T upper, T value)`

The result is `limit_nonnegative_bits(mode, upper, value)`.

`template<typename T> T limit_signed_bits(overflow mode, T upper, T value)`

The result is *overflow*`(mode, -(`$2^{\texttt{upper}}$`-1), `$2^{\texttt{upper}}$`-1, value)`.

`template<overflow mode, typename T> T limit_signed_bits(T upper, T value)`

The result is `limit_signed_bits(mode, upper, value)`.

`template<typename T> T limit_twoscomp_bits(overflow mode, T upper, T value)`

The result is *overflow*`(mode, -`$2^{\texttt{upper}}$`, `$2^{\texttt{upper}}$`-1, value)`.

`template<overflow mode, typename T> T limit_twoscomp_bits(T upper, T value)`

The result is `limit_twoscomp_bits(mode, upper, value)`.

Embedding overflow detection within regular operations can lead to enhanced performance. In particular, left shift is a important candidate operation within fixed-point arithmetic.

`template<typename T> T scale_up(overflow mode, T value, int count)`

The result is *overflow*`(mode, numeric_limits<T>::min, numeric_limits<T>::max, value×`$2^{\texttt{count}}$`)`.

`template<overflow mode, typename T> T scale_up(T value, int count)`

The result is `scale_up(mode, value, count)`.

As before, finer specification of the limits is reasonable.

We can add other functions as needed.

Both Rounding and Overflow

Some operations may reasonably both require rounding and require overflow detection.

First and foremost, conversion from floating-point to integer may require handling a floating-point value that has both a finer resolution and a larger range than the integer can handle. The problem generalizes to arbitrary numeric types.

`template<typename T, typename U> T convert(overflow omode, rounding rmode, U value)`

> The result is *overflow*`(omode, numeric_limits<T>::min, numeric_limits<T>::max,` *round*`(rmode,value))`.

`template<overflow omode, rounding rmode, typename T, typename U> T convert(U value)`

> The result is `convert(omode, rmode; value)`.

Consider shifting as multiplication by a power of two. It has an analogy in a *bidirectional* shift, where a positive power is a left shift and a negative power is a right shift.

`template<typename T> T scale(overflow omode, rounding rmode, T value, int count)`

> The result is `count` < 0
> ? *round*`(rmode,value`$\times 2^{\texttt{count}}$`)`
> : *overflow*`(omode, numeric_limits<T>::min, numeric_limits<T>::max, value`$\times 2^{\texttt{count}}$`)`.

`template<overflow omode, rounding rmode, typename T> T scale(T value, int count)`

> The result is `scale(omode, rmode value count)`.

General Conversion

Conversion between arbitrary numeric types requires something more practical than implementing the full cross product of conversion possibilities.

To that end, we propose that each numeric type *promotes* to an unbound type in it same general category. For example, integers of fixed size would promote to an unbound integer. In this promotion, there can be no possibility of overflow or rounding. Each type also *demotes* from that type. The demotion may have both round and overflow.

The general template conversion algorithm from type *S* to type *T* is to:

- Promote *S* to its unbound type *S′*.
- Convert *S′* to unbound type *T′* of *T*.
- Demote *T′* to *T*.

We expect common conversions to have specialized implementations.

Revisions

This paper revises N4448 - 2015-04-12.

- Add contents entries.
- Add revisions section.
- Rename `bshift` to `scale`, `lshift` to `scale_up` and `rshift` to `scale_down`.
- Rename `positive` to `nonnegative` in function names.
- Make unmotivated rounding modes conditionally supported.
- Remove unmotivated modular overflow modes.
- Provide two sets of functions, one with modes as function parameters and one with modes as template parameters.
- Remove section Template Parameter versus Function Parameter.
- Add a General Conversion section with algorithm.